НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра математичних методів захисту інформації

Лабораторна робота №2 з дисципліни Методи криптоаналізу II

Роботу виконували:
Юрчук Олексій, ФІ-52мн
Дигас Богдан, ФІ-52мн

10 квітня 2026 р.
м. Київ

ЗМІСТ

1 Intro **1**
1.1 Вхідні дані 1
1.2 Хід виконання роботи 2
1.2.1 Коефіцієнти Уолша-Адамара та афінні статистичні аналоги . . . 2
1.2.2 Вибір аналогів та необхідна кількість матеріалу 3
1.2.3 Оцінка складності 5
1.2.4 Передобчислення послідовностей ЛРЗЗ 5
1.2.5 Результати статистичних атак 5
1.2.6 Перевірка 7
1.3 Висновки 7
1.4 Code listing section 8

Список використаних джерел **22**

1

Intro

$$\text{Варіант} = \max\{5; 10\} = 10$$

Мета комп'ютерного практикуму: Практична реалізація статистичної атаки на комбінувальний генератор гами; набуття навичок роботи з системами комп'ютерної алгебри в SAGE.
Постановка задачі:

1. Обчислити коефіцієнти Уолша-Адамара функції f. За допомогою цих коефіцієнтів знайти всі афінні статистичні аналоги f та ймовірності збігу цих аналогів з f.

2. Вибрати набір статистичних аналогів $g_1, \dots, g_r$, що будуть використані для відновлення початкового стану генератора. Для всіх функцій з цього набору обчислити необхідну кількість матеріалу $T_1, \dots, T_r$ відповідно, зафіксувавши $\delta = 0.01$. Побудувати низку атак на початкові стани регістрів, тим самим повністю відновивши початковий стан генератора гами.

3. Перевірити, що початковий стан генератора відновлено правильно, шляхом генерування відрізка гами відповідної довжини й порівняти його з вхідними даними.

1.1 Вхідні дані

1. Шість лінійних регістрів зсуву довжини $m = 10$ з поліномами зворотного зв'язку є наступними:

$$\begin{aligned}
&p_1(x) = x_{10} \oplus x_3 \oplus 1, \quad p_2(x) = x_{10} \oplus x_7 \oplus 1,\\
&p_3(x) = x_{10} \oplus x_4 \oplus x_3 \oplus x_1 \oplus 1, \quad p_4(x) = x_{10} \oplus x_8 \oplus x_3 \oplus x_2 \oplus 1,\\
&p_5(x) = x_{10} \oplus x_8 \oplus x_4 \oplus x_3 \oplus 1, \quad p_6(x) = x_{10} \oplus x_8 \oplus x_5 \oplus x_1 \oplus 1.
\end{aligned}$$

2. Комбінувальна функція $f(x_0, \dots, x_{63})$ генератора гами у файлі **combinfuncV10.txt**

$$x_0x_2x_4 + x_1x_2x_4 + x_1x_4 + x_2x_3x_4 + x_2x_4x_5 + x_2x_4 + x_2 + x_3x_4 + x_4 + 1$$

   $\deg f = 3$, вага булевої функції – кількість входів, на яких вона дорівнює 1 за таблицею істинності. В нашому випадку $\mathrm{wt}(f) = 32$ (збалансована).

3. Відрізок гами $\gamma_1, \dots, \gamma_N$ довжини $N = 50000$ бітів у файлі **gamma2V10.txt**.

1.2 Хід виконання роботи

Використовував пакет комп'ютерної алгебри SageMath (версія 10.7) на WSL 2 Ubuntu, встановлений при виконанні попередньої лаби.

1.2.1 Коефіцієнти Уолша-Адамара та афінні статистичні аналоги

Перетворення Уолша-Адамара [1] є основним інструментом аналізу кореляційних властивостей булевих функцій. Для функції $f \in B_n$ коефіцієнт Уолша-Адамара в точці $\alpha \in V_n$ визначається як:

$$\hat{f}(\alpha) = \sum_{x \in V_n} (-1)^{f(x) \oplus \langle \alpha, x \rangle},$$

де $\langle \alpha, x \rangle = \bigoplus_{i=0}^{n-1} \alpha_i x_i$ – скалярний добуток у $\mathrm{GF}(2)$.

Кожен ненульовий коефіцієнт $\hat{f}(\alpha)$ визначає афінний статистичний аналог $g = \langle \alpha, x \rangle \oplus c$, де $c = 0$ при $\hat{f}(\alpha) > 0$ та $c = 1$ при $\hat{f}(\alpha) < 0$. В даному випадку c виступає аналогом константи (вільним членом) в афінному представленні. Зміщення (bias) аналога:

$$\varepsilon = \frac{|\hat{f}(\alpha)|}{2^n},$$

а ймовірність збігу f та g:

$$\Pr\left(f(X) = \underbrace{\alpha_1 X_n \oplus \ldots \alpha_n X_n \oplus c}_{g(X)} \right) = \frac{1}{2}\left(1 + (-1)^c \cdot \varepsilon\right).$$

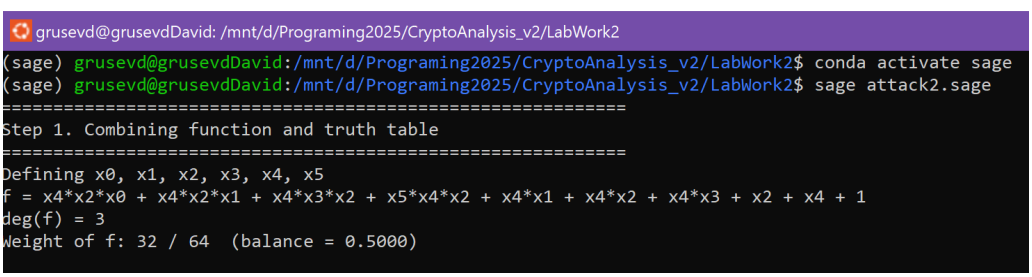


Рис. 1.1: Крок 1: запуск і комбінувальна функція

Коефіцієнти обраховували вбудованою функцією SAGE `walsh_hadamard_transform()`.

```
============================================================
Step 2. Walsh-Hadamard Transform
============================================================
All Walsh-Hadamard coefficients:
  alpha = 000000  (const              ): f_hat =    0
  alpha = 100000  (x0                 ): f_hat =    0
  alpha = 010000  (x1                 ): f_hat =    0
  alpha = 110000  (x0 + x1            ): f_hat =    0
  alpha = 001000  (x2                 ): f_hat =  -32 <-- nonzero
  alpha = 101000  (x0 + x2            ): f_hat =    0
  alpha = 011000  (x1 + x2            ): f_hat =    0
  alpha = 111000  (x0 + x1 + x2       ): f_hat =    0
  alpha = 000100  (x3                 ): f_hat =    0
  alpha = 100100  (x0 + x3            ): f_hat =    0
  alpha = 010100  (x1 + x3            ): f_hat =  +16 <-- nonzero
  alpha = 110100  (x0 + x1 + x3       ): f_hat =    0
  alpha = 001100  (x2 + x3            ): f_hat =    0
  alpha = 101100  (x0 + x2 + x3       ): f_hat =    0
  alpha = 011100  (x1 + x2 + x3       ): f_hat =  +16 <-- nonzero
  alpha = 111100  (x0 + x1 + x2 + x3  ): f_hat =    0
  alpha = 000010  (x4                 ): f_hat =    0
  alpha = 100010  (x0 + x4            ): f_hat =    0
  alpha = 010010  (x1 + x4            ): f_hat =    0
  alpha = 110010  (x0 + x1 + x4       ): f_hat =    0
  alpha = 001010  (x2 + x4            ): f_hat =  -32 <-- nonzero
  alpha = 101010  (x0 + x2 + x4       ): f_hat =    0
  alpha = 011010  (x1 + x2 + x4       ): f_hat =    0
  alpha = 111010  (x0 + x1 + x2 + x4  ): f_hat =    0
  alpha = 000110  (x3 + x4            ): f_hat =    0
  alpha = 100110  (x0 + x3 + x4       ): f_hat =    0
  alpha = 010110  (x1 + x3 + x4       ): f_hat =  -16 <-- nonzero
  alpha = 110110  (x0 + x1 + x3 + x4  ): f_hat =    0
  alpha = 001110  (x2 + x3 + x4       ): f_hat =    0
  alpha = 101110  (x0 + x2 + x3 + x4  ): f_hat =    0
  alpha = 011110  (x1 + x2 + x3 + x4  ): f_hat =  -16 <-- nonzero
  alpha = 111110  (x0 + x1 + x2 + x3 + x4): f_hat =    0
  alpha = 000001  (x5                 ): f_hat =    0
  alpha = 100001  (x0 + x5            ): f_hat =  +16 <-- nonzero
```

Рис. 1.2: Крок 2: коефіцієнти Уолша-Адамара

```
  alpha = 100001  (x0 + x5            ): f_hat =  +16 <-- nonzero
  alpha = 010001  (x1 + x5            ): f_hat =    0
  alpha = 110001  (x0 + x1 + x5       ): f_hat =    0
  alpha = 001001  (x2 + x5            ): f_hat =    0
  alpha = 101001  (x0 + x2 + x5       ): f_hat =  -16 <-- nonzero
  alpha = 011001  (x1 + x2 + x5       ): f_hat =    0
  alpha = 111001  (x0 + x1 + x2 + x5  ): f_hat =    0
  alpha = 000101  (x3 + x5            ): f_hat =    0
  alpha = 100101  (x0 + x3 + x5       ): f_hat =    0
  alpha = 010101  (x1 + x3 + x5       ): f_hat =    0
  alpha = 110101  (x0 + x1 + x3 + x5  ): f_hat =    0
  alpha = 001101  (x2 + x3 + x5       ): f_hat =    0
  alpha = 101101  (x0 + x2 + x3 + x5  ): f_hat =    0
  alpha = 011101  (x1 + x2 + x3 + x5  ): f_hat =    0
  alpha = 111101  (x0 + x1 + x2 + x3 + x5): f_hat =    0
  alpha = 000011  (x4 + x5            ): f_hat =    0
  alpha = 100011  (x0 + x4 + x5       ): f_hat =  -16 <-- nonzero
  alpha = 010011  (x1 + x4 + x5       ): f_hat =    0
  alpha = 110011  (x0 + x1 + x4 + x5  ): f_hat =    0
  alpha = 001011  (x2 + x4 + x5       ): f_hat =    0
  alpha = 101011  (x0 + x2 + x4 + x5  ): f_hat =  +16 <-- nonzero
  alpha = 011011  (x1 + x2 + x4 + x5  ): f_hat =    0
  alpha = 111011  (x0 + x1 + x2 + x4 + x5): f_hat =    0
  alpha = 000111  (x3 + x4 + x5       ): f_hat =    0
  alpha = 100111  (x0 + x3 + x4 + x5  ): f_hat =    0
  alpha = 010111  (x1 + x3 + x4 + x5  ): f_hat =    0
  alpha = 110111  (x0 + x1 + x3 + x4 + x5): f_hat =    0
  alpha = 001111  (x2 + x3 + x4 + x5  ): f_hat =    0
  alpha = 101111  (x0 + x2 + x3 + x4 + x5): f_hat =    0
  alpha = 011111  (x1 + x2 + x3 + x4 + x5): f_hat =    0
  alpha = 111111  (x0 + x1 + x2 + x3 + x4 + x5): f_hat =    0
```

Рис. 1.3: Крок 2: коефіцієнти Уолша-Адамара (continue)

Знайдено 10 ненульових коефіцієнтів:

```
============================================================
Step 3. Affine statistical analogs
============================================================

Analog g                          k      eps  Pr(f=g)  Pr(f!=g)  f_hat
-----------------------------------------------------------------------
  g = x2 + 1                      1   0.5000   0.7500    0.2500    -32
  g = x2 + x4 + 1                 2   0.5000   0.7500    0.2500    -32
  g = x1 + x3                     2   0.2500   0.6250    0.3750    +16
  g = x0 + x5                     2   0.2500   0.6250    0.3750    +16
  g = x1 + x2 + x3                3   0.2500   0.6250    0.3750    +16
  g = x1 + x3 + x4 + 1            3   0.2500   0.6250    0.3750    -16
  g = x0 + x2 + x5 + 1            3   0.2500   0.6250    0.3750    -16
  g = x0 + x4 + x5 + 1            3   0.2500   0.6250    0.3750    -16
  g = x1 + x2 + x3 + x4 + 1       4   0.2500   0.6250    0.3750    -16
  g = x0 + x2 + x4 + x5           4   0.2500   0.6250    0.3750    +16

Total nonzero coefficients: 10

============================================================
Step 4. Correlation immunity
============================================================
cor(f) = 0
  -> f has weight-1 nonzero WHT coefficients => single-register analogs exist
```

Рис. 1.4: Крок 3: g аналоги і $\mathrm{cor}(f)$

Кореляційна імунність [1]: $\mathrm{cor}(f) = 0$, оскільки існує ненульовий коефіцієнт ваги 1 ($\hat{f}(001000) = -32$, що відповідає $g = x_2 \oplus 1$). Це означає наявність аналога від *одного* регістру. На даному screenshot (і далі) під "+" мається на увазі $\oplus$.

1.2.2 Вибір аналогів та необхідна кількість матеріалу

Атака Зігенталера [1] використовує метод максимуму правдоподібності: для кожного набору кандидатів на початкові стани k регістрів генерується відрізок гами довжини T через аналог g і обчислюється відстань Гемінга до відповідного відрізку вхідної гами. Набір з мінімальною такою відстанню є найбільш імовірним початковим станом гами.

Необхідна кількість матеріалу [2] (та й в наших лекціях це було):

$$T = \lceil 8 \cdot \varepsilon^{-2} \cdot \ln(2^{mk}/\delta) \rceil ,$$

де $m = 10$ (довжина регістру), k — кількість регістрів в аналозі, $\delta = 0.01$ – фіксоване.

```
============================================================
Step 5. Reading gamma sequence
============================================================
Gamma length: 50000
Number of 0s: 24735 (49.5%)
Number of 1s: 25265 (50.5%)

============================================================
Step 6. LFSR parameters
============================================================
  R1: x^10 + x^3 + 1                        tap_mask = 9
  R2: x^10 + x^7 + 1                        tap_mask = 129
  R3: x^10 + x^4 + x^3 + x + 1              tap_mask = 27
  R4: x^10 + x^8 + x^3 + x^2 + 1            tap_mask = 269
  R5: x^10 + x^8 + x^4 + x^3 + 1            tap_mask = 281
  R6: x^10 + x^8 + x^5 + x + 1              tap_mask = 291

============================================================
Step 7. Required material T for each analog
============================================================
delta = 0.0100000000000000

Analog g                          k        T            Registers   Candidates
--------------------------------------------------------------------------------
  g = x2 + 1                      1      370                   R3         1023
  g = x2 + x4 + 1                 2      591               R3, R5      1046529
  g = x1 + x3                     2     2364               R2, R4      1046529
  g = x0 + x5                     2     2364               R1, R6      1046529
  g = x1 + x2 + x3                3     3252           R2, R3, R4   1070599167
  g = x1 + x3 + x4 + 1            3     3252           R2, R4, R5   1070599167
  g = x0 + x2 + x5 + 1            3     3252           R1, R3, R6   1070599167
  g = x0 + x4 + x5 + 1            3     3252           R1, R5, R6   1070599167
  g = x1 + x2 + x3 + x4 + 1       4     4139       R2, R3, R4, R5 1095222947841
  g = x0 + x2 + x4 + x5           4     4139       R1, R3, R5, R6 1095222947841
```

Рис. 1.5: Кроки 5–7: зчитування гами, параметри ЛРЗЗ, обчислення T

Стратегія вибору аналогів визначається автоматично за "жадібним" принципом:

- Аналоги g сортуються за зростанням k (менше регістрів — менше кандидатів для перебору) та за спаданням ε (більше зміщення — менше потрібно матеріалу T).
- На кожному етапі обирається найдешевший аналог, що покриває лише ще невідновлені регістри.
- Якщо для якогось регістру не залишилось "чистого" аналога (всі аналоги, що його містять, мають хоча б один уже відновлений регістр), цей регістр відновлюватиметься повним перебором.

Для нашого варіанту ця процедура дала наступну послідовність атак:

1. $g_1 = x_2 \oplus 1$ ($k = 1$, $\varepsilon = 0.5$, регістр R3): $T_1 = 370$ бітів, 1023 кандидати.
2. $g_2 = x_1 \oplus x_3$ ($k = 2$, $\varepsilon = 0.25$, регістри R2, R4): $T_2 = 2364$ бітів, $1\,046\,529$ кандидатів.
3. $g_3 = x_0 \oplus x_5$ ($k = 2$, $\varepsilon = 0.25$, регістри R1, R6): $T_3 = 2364$ бітів, $1\,046\,529$ кандидатів.
4. Регістр R5 відновлюється повним перебором (1023 кандидати).

Зауваження 1. R5 потрапив у повний перебір, оскільки єдиний аналог з $k \leq 2$, що його містить — $g = x_2 \oplus x_4 \oplus 1$ (R3, R5) – має R3 вже відновленим на першому етапі, а

написаний скрипт пропускає аналоги з частково відомими регістрами, бо я подумав що, при $m = 10$ перебрати 1023 кандидатів швидше, ніж ускладнювати логіку для часткової атаки, хех. І це справді виявилося так:) (див. час роботи нижче)

Зауваження 2. Ще варто зазначити, що якби функція f мала вищу кореляційну імунність (наприклад, $\mathrm{cor}(f) = 2$), то аналогів від одного регістру не існувало б, і той самий алгоритм автоматично почав би з аналогів при $k \geq 3$.

1.2.3 Оцінка складності

Згідно з [1] (і конспектом), часова складність атаки з аналогом від k регістрів становить:

$$O\left(T \cdot 2^{mk} + nm \cdot 2^{m(n-k)}\right),$$

де перший доданок – складність статистичної фази (перебір 2^{mk} кандидатів, для кожного порівняння T бітів), а другий – складність повного перебору залишкових $n-k$ регістрів. Для нашого варіанту ($n = 6$, $m = 10$):

Етап	k	$T \cdot 2^{mk}$	$nm \cdot 2^{m(n-k)}$	**Сумарно**
$g_1 = x_2 \oplus 1$	1	$370 \cdot 2^{10} \approx 3.8 \cdot 10^5$	$60 \cdot 2^{50} \approx 6.8 \cdot 10^{16}$	$\approx 6.8 \cdot 10^{16}$
$g_2 = x_1 \oplus x_3$	2	$2364 \cdot 2^{20} \approx 2.5 \cdot 10^9$	$60 \cdot 2^{40} \approx 6.6 \cdot 10^{13}$	$\approx 6.6 \cdot 10^{13}$
$g_3 = x_0 \oplus x_5$	2	$2364 \cdot 2^{20} \approx 2.5 \cdot 10^9$	$60 \cdot 2^{40} \approx 6.6 \cdot 10^{13}$	$\approx 6.6 \cdot 10^{13}$

Однак атаки виконуються послідовно: після етапу 1 відомий стан R3, тому на етапі 2 перебираються лише 2 невідомих регістри (а не $n-k = 4$ залишкових повним перебором). Аналогічно після етапів 1–3 залишається лише R5, для якого перебір $2^{10} - 1 = 1023$ кандидатів є тривіальним.
Фактична складність трьох статистичних фаз:

$$T_1 \cdot 2^m + T_2 \cdot 2^{2m} + T_3 \cdot 2^{2m} + 2^m \approx 3.8 \cdot 10^5 + 2.5 \cdot 10^9 + 2.5 \cdot 10^9 + 10^3 \approx 5 \cdot 10^9, \tag{1.1}$$

що на багато порядків менше за повний перебір $2^{nm} = 2^{60} \approx 1.15 \cdot 10^{18}$.

1.2.4 Передобчислення послідовностей ЛРЗЗ

Перед безпосереднім запуском атак виконується передобчислення: для кожного з 6 регістрів і кожного з $2^m - 1 = 1023$ можливих ненульових початкових станів – генерується вихідна послідовність ЛРЗЗ довжиною $T_{\max}$ (максимальне T серед усіх обраних аналогів).

Кожна послідовність зберігається як одне велике ціле число (`big integer`), де біт i дорівнює виходу ЛРЗЗ у момент часу i. Всього обчислюється $6 \times 1023 = 6138$ послідовностей.

Це дозволяє обчислювати відстань Гемінга між гамою та виходом аналога за допомогою лише двох побітових операцій: XOR двох великих чисел та підрахунок одиничних бітів (`popcount`). Без передобчислення довелося б на кожному з мільйонів кандидатів заново генерувати послідовність ЛРЗЗ побітово, що очевидно було б повільніше (тому я навіть цим і не заморочувався). Час передобчислення склав ≈ 20 с.

1.2.5 Результати статистичних атак

Етап 1: $g_1 = x_2 \oplus 1$ **(регістр R3).** Перебір 1023 кандидатів, $T_1 = 370$ бітів. Мінімальне значення статистики (абсолютне): 84, відносне: $84/370 = 0.2270$. Друге найкраще (абсолютне): 156, відносне: $156/370 = 0.4216$. Час: < 0.01 с. **R3 = 972** (`0b1111001100`).

```
============================================================
Step 8. Precomputing LFSR sequences
============================================================
Sequence length: 4141
States per register: 1023
Precomputation time: 20.12 s
Total sequences: 6138
```

Рис. 1.6: Крок 8: передобчислення послідовностей LFSR

Етап 2: $g_2 = x_1 \oplus x_3$ **(регістри R2, R4).** Перебір $1\,046\,529$ кандидатів, $T_2 = 2364$ бітів. Мінімальне (абс.): 899, відносне: $899/2364 = 0.3803$. Друге найкраще (абс.): 1012, відносне: $1012/2364 = 0.4281$. Час: ≈ 11.4 с. **R2 = 610** (`0b1001100010`), **R4 = 272** (`0b0100010000`).

Етап 3: $g_3 = x_0 \oplus x_5$ **(регістри R1, R6).** Перебір $1\,046\,529$ кандидатів, $T_3 = 2364$ бітів. Мінімальне (абс.): 873, відносне: $873/2364 = 0.3693$. Друге найкраще (абс.): 998, відносне: $998/2364 = 0.4222$. Час: ≈ 11.2 с. **R1 = 223** (`0b0011011111`), **R6 = 672** (`0b1010100000`).

Етап 4: повний перебір R5. Перебір 1023 кандидатів (із зафіксованими R1–R4, R6), $T = 2000$ бітів. Мінімальне (абс.): 0 (точний збіг). Час: ≈ 12.9 с. **R5 = 146** (`0b0010010010`).

```
============================================================
Step 9. Statistical attacks
============================================================

--- Attacking with g = x2 + 1 ---
    Registers: [3], k=1, T=370, eps=0.5000
    [offset=0] time=0.01s  best=84/370 (0.2270)  2nd_best=156/370 (0.4216)  expected_true~92  expected_false~185
    [offset=1] time=0.00s  best=84/370 (0.2270)  2nd_best=156/370 (0.4216)  expected_true~92  expected_false~185
    => Determined working offset = 0
    => R3: initial state = 972 (0b1111001100)

--- Attacking with g = x1 + x3 ---
    Registers: [2, 4], k=2, T=2364, eps=0.2500
    [offset=0] time=11.39s  best=899/2364 (0.3803)  2nd_best=1012/2364 (0.4281)  expected_true~886  expected_false~1182
    => R2: initial state = 610 (0b1001100010)
    => R4: initial state = 272 (0b0100010000)

--- Attacking with g = x0 + x5 ---
    Registers: [1, 6], k=2, T=2364, eps=0.2500
    [offset=0] time=11.16s  best=873/2364 (0.3693)  2nd_best=998/2364 (0.4222)  expected_true~886  expected_false~1182
    => R1: initial state = 223 (0b0011011111)
    => R6: initial state = 672 (0b1010100000)

--- Recovered so far: 5 / 6 registers ---
  R1: 223 (0b0011011111)
  R2: 610 (0b1001100010)
  R3: 972 (0b1111001100)
  R4: 272 (0b0100010000)
  R6: 672 (0b1010100000)

============================================================
Step 10. Brute-forcing remaining registers: [5]
============================================================
Candidates to try: 1023
Brute-force time: 12.84 s
Best Hamming distance: 0 / 2000
  R5: 146 (0b0010010010)

*** All recovered initial states: ***
  R1:  223 (0b0011011111)
  R2:  610 (0b1001100010)
  R3:  972 (0b1111001100)
  R4:  272 (0b0100010000)
  R5:  146 (0b0010010010)
  R6:  672 (0b1010100000)
```

Рис. 1.7: Кроки 9–10: статистичні атаки, повний перебір R5

Зауваження. При повному переборі R5 ми вже знаємо стани всіх інших 5 регістрів. Тому для кожного кандидата на R5 генеруємо гаму через повну комбінувальну функцію f (а не через наближений аналог g). Коли кандидат правильний, тобто згенерована гама точно збігається з вхідною, тому відстань Гемінга = 0.

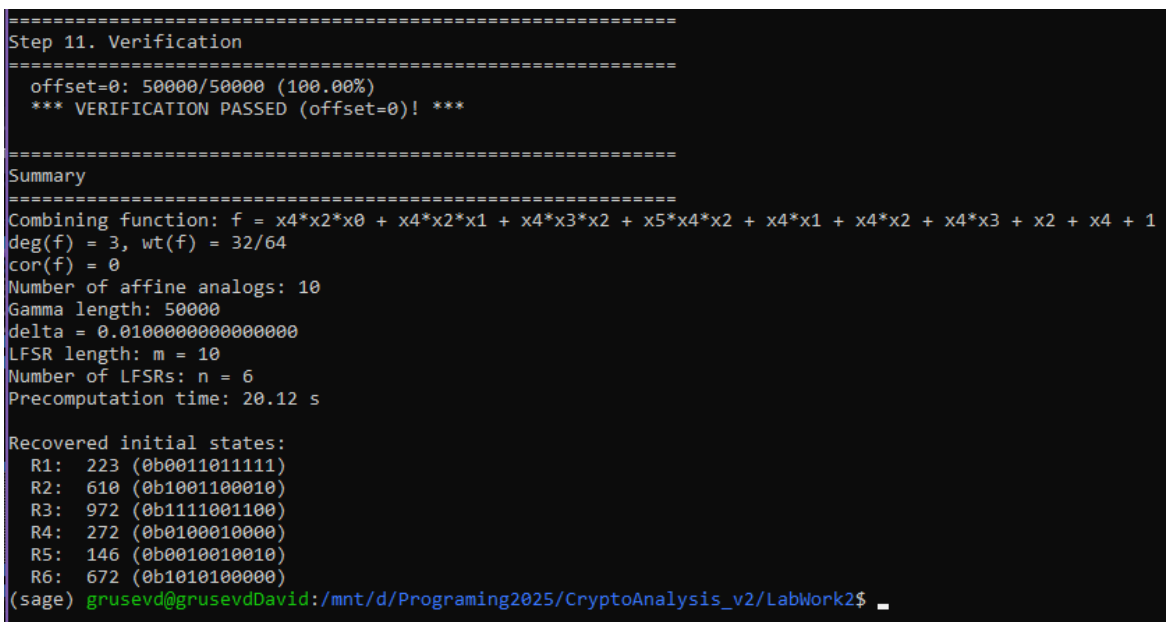
Отже, відновлений початковий стан генератора має вид:

Регістр	decimal	binary
R1	223	0011011111
R2	610	1001100010
R3	972	1111001100
R4	272	0100010000
R5	146	0010010010
R6	672	1010100000

1.2.6 Перевірка

Зі знайдених початкових станів було згенеровано $N = 50000$ бітів гами за допомогою повної комбінувальної функції f та порівняно з вхідними даними.

Результат як очікувано – усі 50000 з 50000 бітів збіглися (**100%**) – перемога!.



```
============================================================
Step 11. Verification
============================================================
  offset=0: 50000/50000 (100.00%)
  *** VERIFICATION PASSED (offset=0)! ***

============================================================
Summary
============================================================
Combining function: f = x4*x2*x0 + x4*x2*x1 + x4*x3*x2 + x5*x4*x2 + x4*x1 + x4*x2 + x4*x3 + x2 + x4 + 1
deg(f) = 3, wt(f) = 32/64
cor(f) = 0
Number of affine analogs: 10
Gamma length: 50000
delta = 0.0100000000000000
LFSR length: m = 10
Number of LFSRs: n = 6
Precomputation time: 20.12 s

Recovered initial states:
  R1:  223 (0b0011011111)
  R2:  610 (0b1001100010)
  R3:  972 (0b1111001100)
  R4:  272 (0b0100010000)
  R5:  146 (0b0010010010)
  R6:  672 (0b1010100000)
(sage) grusevd@grusevdDavid:/mnt/d/Programing2025/CryptoAnalysis_v2/LabWork2$
```

Рис. 1.8: Крок 11: верифікація та підсумок

1.3 Висновки

У ході роботи ми реалізовували статистичну атаку Зігенталера на комбінувальний генератор гами, де $n = 6$ регістрів довжини $m = 10$, $\deg f = 3$

1. Обчислили коефіцієнти Уолша-Адамара функції f. Знайдено 10 афінних статистичних аналогів. Кореляційна імунність склала $\mathrm{cor}(f) = 0$, що означає наявність аналога від одного регістру.

2. Для відновлення станів використано 3 аналоги ($k = 1, 2, 2$) та повний перебір для одного залишкового регістру. Загальний час атаки: $< 0.01 + 11.4 + 11.2 + 12.9 \approx 36.5$ с (без передобчислення).

3. Відновлено початкові стани всіх 6 регістрів:

$$R1 = 223,\ R2 = 610,\ R3 = 972,\ R4 = 272,\ R5 = 146,\ R6 = 672.$$

4. Початковий стан перевірено шляхом генерування гами, всі $N = 50000$ бітів збіглися з вхідними даними.

Низька кореляційна імунність ($\mathrm{cor}(f) = 0$) зробила атаку надзвичайно ефективною: наявність аналога від одного регістру дозволила відновити R3 за частки секунди. Загальна складність атаки $\approx 5 \cdot 10^9$ операцій (1.1), що на 9 порядків менше за повний перебір $2^{60} \approx 1.15 \cdot 10^{18}$ можливих станів генератора [2].

1.4 Code listing section

Лістинг 1.1: Статистична атака на комбінувальний генератор Gamma з використанням SAGE

```
#!/usr/bin/env sage
# -*- coding: utf-8 -*-
"""
Statistical attack on a combining gamma generator

Attack outline (Siegenthaler's attack):
  1) Compute Walsh-Hadamard coefficients of combining function f
  2) Find ALL affine statistical analogs of f and their coincidence probabilities
  3) Compute correlation immunity cor(f)
  4) For each chosen analog g (depends on k out of n registers):
     - Compute required material T (with delta = 0.01)
     - Enumerate all 2^(mk) candidate initial states for those k registers
     - For each candidate, generate T bits via g, compare with gamma (Hamming distance)
     - The candidate with minimum Hamming distance is the true initial state
  5) Recover all 6 registers (possibly using multiple analogs + brute force)
  6) Verify by generating gamma from the full recovered state
"""

import time
import re
import math

# IMPORTANT: While sage preparsing converts ^ to ** (exponentiation) and ^^ to ^.
# To avoid any issues, we define explicit XOR and popcount functions
# using Python's int methods directly.
def bxor(a, b):
    """Bitwise XOR, safe from Sage's ^ preparser."""
    return int.__xor__(int(a), int(b))

def popcount(x):
    """Count number of 1-bits in integer x."""
```

```
    return bin(int(x)).count('1')

# ============================================================
# Step 1. Parse the combining function and build truth table
# ============================================================
# The combining function f(x0,...,x5) acts on outputs of 6 LFSRs.
# It is defined over GF(2) - all arithmetic is mod 2 (XOR for +, AND for *).
# We build a truth table: for each of the 2^6 = 64 input combinations,
# store f(x0,...,x5) in a lookup array. This enables O(1) evaluation later.

print("=" * 60)
print("Step 1. Combining function and truth table")
print("=" * 60)

n = 6   # number of LFSRs / variables in f
m = 10  # length of each LFSR (bits)

# Parse the combining function from fileS
with open('combinfuncV10.txt', 'r') as file:
    func_str = file.read().strip()

# Use Sage's BooleanPolynomialRing for symbolic representation
B = BooleanPolynomialRing(n, 'x', order='degrevlex')
B.inject_variables()
xvars = B.gens()

# Parse: split by " + ", extract x_{i} indices, build polynomial
terms = func_str.split(' + ')
f_poly = B(0)
for term in terms:
    term = term.strip()
    if term == '1':
        f_poly += B(1)
        continue
    var_indices = re.findall(r'x_\{(\d+)\}', term)
    if not var_indices:
        continue
    monomial = B(1)
    for v in var_indices:
        monomial *= xvars[int(v)]
    f_poly += monomial

print(f"f = {f_poly}")
print(f"deg(f) = {f_poly.degree()}")

# Build truth table using Sage's BooleanFunction
# f_tt[i] = f(bit_0(i), bit_1(i), ..., bit_5(i))
# where i = x0 + 2*x1 + 4*x2 + 8*x3 + 16*x4 + 32*x5
from sage.crypto.boolean_function import BooleanFunction
bf = BooleanFunction(f_poly)
f_tt = list(bf.truth_table(format='int'))  # list of 0/1 of length 64
```

```python

wt_f = sum(f_tt)
print(f"Weight of f: {wt_f} / {2**n}  (balance = {float(wt_f / 2**n):.4f})")

# ============================================================
# Step 2. Walsh-Hadamard Transform
# ============================================================
# The Walsh-Hadamard coefficient f_hat(alpha) is defined as:
#   f_hat(alpha) = sum_{x in V_n} (-1)^{f(x) XOR <alpha, x>}
# where <alpha, x> = XOR of (alpha_i AND x_i) is the inner product mod 2.
#
# These coefficients reveal the correlation between f and each affine function.
# Sage's BooleanFunction computes this directly.

print("\n" + "=" * 60)
print("Step 2. Walsh-Hadamard Transform")
print("=" * 60)

wh = list(bf.walsh_hadamard_transform())

print("All Walsh-Hadamard coefficients:")
for alpha in range(2**n):
    bits = ''.join(str((alpha >> j) & 1) for j in range(n))
    vars_str = ' + '.join(f'x{j}' for j in range(n) if (alpha >> j) & 1)
    if not vars_str:
        vars_str = "const"
    status = f"f_hat = {wh[alpha]:+4d}" if wh[alpha] != 0 else "f_hat =    0"
    marker = " <-- nonzero" if wh[alpha] != 0 else ""
    print(f"  alpha = {bits}  ({vars_str:20s}): {status}{marker}")

# ============================================================
# Step 3. Affine statistical analogs and coincidence probabilities
# ============================================================
# For each alpha with f_hat(alpha) != 0:
#   - If f_hat(alpha) > 0: the analog is g = <alpha, x>       (c = 0)
#   - If f_hat(alpha) < 0: the analog is g = <alpha, x> + 1   (c = 1)
#   - Coincidence probability: Pr(f = g) = 1/2 * (1 + |f_hat(alpha)| / 2^n)
#   - The "epsilon" (bias) is: eps = |f_hat(alpha)| / 2^n
#   - Error probability: p = Pr(f != g) = 1/2 * (1 - eps)
#
# The analog g depends on k = wt(alpha) variables (registers).
# The fewer variables, then attack is "cheaper" (fewer candidates to enumerate).

print("\n" + "=" * 60)
print("Step 3. Affine statistical analogs")
print("=" * 60)

analogs = []
for alpha in range(2**n):
```

```python
    fh = wh[alpha]
    if fh == 0:
        continue

    c = 0 if fh > 0 else 1
    eps = float(abs(fh)) / float(2**n)
    prob = 0.5 * (1.0 + eps)    # Pr(f = g)
    err = 0.5 * (1.0 - eps)     # Pr(f != g)

    var_indices = [j for j in range(n) if (alpha >> j) & 1]
    k = len(var_indices)

    # Build human-readable string for g
    if k == 0:
        g_str = str(c)
    else:
        parts = [f"x{j}" for j in var_indices]
        g_str = " + ".join(parts)
        if c == 1:
            g_str += " + 1"

    analogs.append({
        'alpha': alpha, 'c': c, 'fh': fh, 'eps': eps,
        'prob': prob, 'err': err,
        'vars': var_indices, 'k': k, 'g_str': g_str
    })

print(f"\n{'Analog g':30s} {'k':>3s} {'eps':>8s} {'Pr(f=g)':>8s} {'Pr(f!=g)':>9s} {'f_hat':>6s}")
print("-" * 70)
for a in sorted(analogs, key=lambda x: (x['k'], -x['eps'])):
    print(f"  g = {a['g_str']:24s} {a['k']:3d} {a['eps']:8.4f} "
          f"{a['prob']:8.4f} {a['err']:9.4f} {a['fh']:+6d}")
print(f"\nTotal nonzero coefficients: {len(analogs)}")

# ============================================================
# Step 4. Correlation immunity
# ============================================================
# cor(f) = max k such that f_hat(alpha) = 0 for every nonzero alpha
# with Hamming weight wt(alpha) <= k.
# If no such k exists, cor(f) = 0.
#
# High correlation immunity means no low-weight analogs exist,
# making the attack harder (must enumerate more registers simultaneously).

print("\n" + "=" * 60)
print("Step 4. Correlation immunity")
print("=" * 60)

cor_f = 0
for k_test in range(1, n + 1):
```

```python
    all_zero = True
    for alpha in range(1, 2**n):
        if bin(alpha).count('1') <= k_test and wh[alpha] != 0:
            all_zero = False
            break
    if all_zero:
        cor_f = k_test
    else:
        break

print(f"cor(f) = {cor_f}")
if cor_f == 0:
    print("  -> f has weight-1 nonzero WHT coefficients => single-register analogs exist")
else:
    print(f"  -> minimum weight of a useful analog is {cor_f + 1}")

# ============================================================
# Step 5. Read the gamma sequence
# ============================================================
print("\n" + "=" * 60)
print("Step 5. Reading gamma sequence")
print("=" * 60)

with open('gamma2V10.txt', 'r') as file:
    gamma_str = file.read().strip()
gamma = [int(b) for b in gamma_str]
N = len(gamma)
n_zeros = gamma.count(0)
n_ones = gamma.count(1)
print(f"Gamma length: {N}")
print(f"Number of 0s: {n_zeros} ({float(100*n_zeros/N):.1f}%)")
print(f"Number of 1s: {n_ones} ({float(100*n_ones/N):.1f}%)")

# ============================================================
# Step 6. LFSR definitions
# ============================================================
# Each LFSR has length m = 10 with a given feedback polynomial.
# The state is represented as a 10-bit integer.
# At each step: output = LSB (bit 0), feedback = XOR of tap positions,
# shift right, insert feedback at MSB (bit 9).
#
# The "tap mask" encodes which coefficients c_i are nonzero in
# p(x) = x^10 + c_9*x^9 + ... + c_1*x + c_0.
# For p(x) = x^10 + x^3 + 1: c_0 = 1, c_3 = 1 => tap_mask = (1<<0)|(1<<3) = 9.

print("\n" + "=" * 60)
print("Step 6. LFSR parameters")
print("=" * 60)

lfsr_info = [
```

```python
    ("R1", "x^10 + x^3 + 1",                (1 << 0) | (1 << 3)),
    ("R2", "x^10 + x^7 + 1",                (1 << 0) | (1 << 7)),
    ("R3", "x^10 + x^4 + x^3 + x + 1",     (1 << 0) | (1 << 1) | (1 << 3) | (1 << 4)),
    ("R4", "x^10 + x^8 + x^3 + x^2 + 1",   (1 << 0) | (1 << 2) | (1 << 3) | (1 << 8)),
    ("R5", "x^10 + x^8 + x^4 + x^3 + 1",   (1 << 0) | (1 << 3) | (1 << 4) | (1 << 8)),
    ("R6", "x^10 + x^8 + x^5 + x + 1",     (1 << 0) | (1 << 1) | (1 << 5) | (1 << 8)),
]

tap_masks = []
for name, poly_str, tap_mask in lfsr_info:
    tap_masks.append(tap_mask)
    print(f"  {name}: {poly_str:35s}  tap_mask = {tap_mask}")

# ============================================================
# Step 7. Compute required material T for each analog
# ============================================================
# From the theory (Proposition 1):
#   T = ceil(8 * eps^{-2} * ln(2^{mk} / delta))
# where:
#   m = 10 (register length), k = number of registers in the analog,
#   delta = 0.01 (error probability of the attack),
#   eps = |f_hat(alpha)| / 2^n (bias of the analog).

print("\n" + "=" * 60)
print("Step 7. Required material T for each analog")
print("=" * 60)

delta = 0.01

for a in analogs:
    k = a['k']
    eps = a['eps']
    if k == 0:
        a['T'] = 0
        continue
    T_val = math.ceil(8 * eps**(-2) * math.log(2**(m * k) / delta))
    a['T'] = min(T_val, N)

print(f"delta = {delta}")
print(f"\n{'Analog g':30s} {'k':>3s} {'T':>8s} {'Registers':>20s} {'Candidates':>12s}")
print("-" * 80)
for a in sorted(analogs, key=lambda x: (x['k'], x.get('T', 0))):
    if a['k'] == 0:
        continue
    reg_str = ', '.join(f"R{j+1}" for j in a['vars'])
    cand = (2**m - 1) ** a['k']
```

```python
    print(f"  g = {a['g_str']:24s} {a['k']:3d} {a['T']:8d} {reg_str:>20s} {cand:>12d}")

# ============================================================
# Step 8. Precompute LFSR sequences
# ============================================================
# For efficiency, we precompute the output sequence of each LFSR for
# every possible nonzero initial state (1..1023). Each sequence is stored
# as a big Python integer where bit i = output at time i.
# This allows ultra-fast Hamming distance computation via XOR + popcount.

print("\n" + "=" * 60)
print("Step 8. Precomputing LFSR sequences")
print("=" * 60)

max_T = max((a['T'] for a in analogs if a['k'] > 0), default=N)
# Add margin for possible offset issues; cap at N
seq_len = min(max_T + 2, N + 2)

def gen_lfsr_bigint(init_state, tap_mask, length):
    """Generate LFSR sequence as a big integer (bit i = output at time i).

    At each step:
      - Output the LSB (s_0)
      - Compute feedback = XOR of bits at tap positions
      - Shift right, insert feedback at MSB position (bit m-1)

    The sequence s_0, s_1, s_2, ... corresponds to successive outputs.
    """
    state = init_state
    mask_m = (1 << m) - 1
    result = 0
    for i in range(length):
        if state & 1:
            result |= (1 << i)
        fb = bin(state & tap_mask).count('1') & 1
        state = ((state >> 1) | (fb << (m - 1))) & mask_m
    return result

print(f"Sequence length: {seq_len}")
print(f"States per register: {2**m - 1}")
t0 = time.time()

# lfsr_seqs[j][s] = big integer of sequence for register j, initial state s
lfsr_seqs = []
for j in range(n):
    seqs = {}
    tap = tap_masks[j]
    for s in range(1, 2**m):
        seqs[s] = gen_lfsr_bigint(s, tap, seq_len)
    lfsr_seqs.append(seqs)
```

```python

t_precomp = time.time() - t0
print(f"Precomputation time: {t_precomp:.2f} s")
print(f"Total sequences: {n * (2**m - 1)}")

# ============================================================
# Step 9. Run statistical attacks
# ============================================================
# Strategy:
#   - Try analogs with k=1 first (cheapest: 1023 candidates each)
#   - Then k=2 if needed (1023^2 \approx 1M candidates)
#   - Brute-force any remaining registers
#
# For each analog g depending on registers i1, ..., ik:
#   g(t) = seq_{i1}(t) XOR seq_{i2}(t) XOR ... XOR c
#   As big integers: g_int = seq_i1 XOR seq_i2 XOR ... XOR c_mask
#   Hamming distance = popcount(g_int XOR gamma_int)
#
# We try TWO offset conventions (gamma might start from LFSR time 0 or 1):
#   offset=0: gamma[k] uses LFSR output at time k
#   offset=1: gamma[k] uses LFSR output at time k+1
# The one that gives a clear separation (best << T/2) is correct.

print("\n" + "=" * 60)
print("Step 9. Statistical attacks")
print("=" * 60)

recovered = {}  # register_index -> initial_state

# Build gamma as big integer (bit i = gamma[i])
gamma_int = 0
for i in range(N):
    if gamma[i]:
        gamma_int |= (1 << i)

# Sort analogs: smallest k first, then largest eps (best quality)
attack_order = sorted(
    [a for a in analogs if a['k'] > 0],
    key=lambda x: (x['k'], -x['eps'])
)

working_offset = None  # will be determined from first successful attack

for a in attack_order:
    var_indices = a['vars']

    # Skip if all registers in this analog are already recovered
    unrecovered = [j for j in var_indices if j not in recovered]
    if len(unrecovered) == 0:
        continue
```

```
    # For simplicity, only attack when ALL registers in this analog are unknown
    if len(unrecovered) != a['k']:
        continue

    k = a['k']
    T = a['T']
    c = a['c']
    eps = a['eps']

    print(f"\n--- Attacking with g = {a['g_str']} ---")
    print(f"    Registers: {[j+1 for j in var_indices]}, k={k}, T={T}, eps={eps:.4f}")

    c_mask_full = (1 << seq_len) - 1 if c == 1 else 0  # all-ones if c=1, else 0

    best_result = None  # will hold (dist, states_dict, offset_used)

    # Try both offsets (unless already determined)
    offsets_to_try = [working_offset] if working_offset is not None else [0, 1]

    for offset in offsets_to_try:
        T_mask = (1 << T) - 1
        gamma_seg = (gamma_int >> offset) & T_mask  # gamma bits [offset, offset+T)
        c_mask = (c_mask_full >> offset) & T_mask

        best_dist = T + 1
        best_states = None
        second_best = T + 1

        t0 = time.time()

        if k == 1:
            j = var_indices[0]
            for s in range(1, 2**m):
                seq = (lfsr_seqs[j][s] >> offset) & T_mask
                g_int = bxor(seq, c_mask)
                dist = popcount(bxor(g_int, gamma_seg))
                if dist < best_dist:
                    second_best = best_dist
                    best_dist = dist
                    best_states = {j: s}
                elif dist < second_best:
                    second_best = dist

        elif k == 2:
            j1, j2 = var_indices
            for s1 in range(1, 2**m):
```

```
                seq1 = (lfsr_seqs[j1][s1] >> offset) & T_mask
                for s2 in range(1, 2**m):
                    seq2 = (lfsr_seqs[j2][s2] >> offset) & T_mask
                    g_int = bxor(bxor(seq1, seq2), c_mask)
                    dist = popcount(bxor(g_int, gamma_seg))
                    if dist < best_dist:
                        second_best = best_dist
                        best_dist = dist
                        best_states = {j1: s1, j2: s2}
                    elif dist < second_best:
                        second_best = dist

        elif k == 3:
            j1, j2, j3 = var_indices
            for s1 in range(1, 2**m):
                seq1 = (lfsr_seqs[j1][s1] >> offset) & T_mask
                for s2 in range(1, 2**m):
                    seq12 = bxor(seq1, (lfsr_seqs[j2][s2] >> offset) & T_mask)
                    for s3 in range(1, 2**m):
                        seq3 = (lfsr_seqs[j3][s3] >> offset) & T_mask
                        g_int = bxor(bxor(seq12, seq3), c_mask)
                        dist = popcount(bxor(g_int, gamma_seg))
                        if dist < best_dist:
                            second_best = best_dist
                            best_dist = dist
                            best_states = {j1: s1, j2: s2, j3: s3}
                        elif dist < second_best:
                            second_best = dist

        t_attack = time.time() - t0

        expected_true = T * a['err']   # expected Hamming distance for correct state
        expected_false = T * 0.5       # expected for wrong state

        print(f"    [offset={offset}] time={t_attack:.2f}s  "
              f"best={best_dist}/{T} ({float(best_dist/T):.4f})  "
              f"2nd_best={second_best}/{T} ({float(second_best/T):.4f})  "
              f"expected_true~{expected_true:.0f}  expected_false~{expected_false:.0f}")

        # Accept if best is clearly better than second best and close to expected
        if best_dist < expected_false * 0.8:
            if best_result is None or best_dist < best_result[0]:
                best_result = (best_dist, best_states, offset)

    if best_result is not None:
        dist, states, off = best_result
        if working_offset is None:
            working_offset = off
```

```
            print(f"    => Determined working offset = {working_offset}")
        for j, s in states.items():
            recovered[j] = s
            print(f"    => R{j+1}: initial state = {s} (0b{s:010b})")
    else:
        print(f"    => Attack inconclusive (no clear winner)")

print(f"\n--- Recovered so far: {len(recovered)} / {n} registers ---")
for j in sorted(recovered.keys()):
    print(f"  R{j+1}: {recovered[j]} (0b{recovered[j]:010b})")

# ============================================================
# Step 10. Brute-force remaining registers
# ============================================================
# If some registers are still unknown, recover them by brute force:
# enumerate all candidate states, generate gamma using full f, compare.
# With recovered registers fixed, each candidate only varies the unknowns.

unrecovered_regs = [j for j in range(n) if j not in recovered]

if unrecovered_regs:
    print(f"\n{'='*60}")
    print(f"Step 10. Brute-forcing remaining registers: {[j+1 for j in
       unrecovered_regs]}")
    print(f"{'='*60}")

    num_unknown = len(unrecovered_regs)
    total_candidates = (2**m - 1) ** num_unknown
    print(f"Candidates to try: {total_candidates}")

    # Use first T_brute bits for verification
    T_brute = min(2000, N)
    off = working_offset if working_offset is not None else 1

    gamma_seg_brute = 0
    for i in range(T_brute):
        if gamma[i] == 1:
            gamma_seg_brute |= (1 << i)

    best_dist_bf = T_brute + 1
    best_states_bf = None
    t0 = time.time()

    # Build candidate iterator
    from itertools import product as iprod
    for combo in iprod(*(range(1, 2**m) for _ in unrecovered_regs)):
        # Set up all register states
        all_states = dict(recovered)
        for idx, j in enumerate(unrecovered_regs):
            all_states[j] = combo[idx]

```

```python
        # Generate gamma using full f and truth table lookup
        gen_int = 0
        reg_states = [all_states[j] for j in range(n)]
        states_copy = list(reg_states)
        mask_m = (1 << m) - 1

        for t_idx in range(off + T_brute):
            if t_idx >= off:
                # Build input to f: bit j = LSB of register j's state
                f_input = 0
                for j in range(n):
                    if states_copy[j] & 1:
                        f_input |= (1 << j)
                if f_tt[f_input]:
                    gen_int |= (1 << (t_idx - off))

            # Clock all registers
            for j in range(n):
                fb = bin(states_copy[j] & tap_masks[j]).count('1') & 1
                states_copy[j] = ((states_copy[j] >> 1) | (fb << (m - 1))) & mask_m

        dist = popcount(bxor(gen_int, gamma_seg_brute))
        if dist < best_dist_bf:
            best_dist_bf = dist
            best_states_bf = dict(all_states)

    t_bf = time.time() - t0
    print(f"Brute-force time: {t_bf:.2f} s")
    print(f"Best Hamming distance: {best_dist_bf} / {T_brute}")

    if best_states_bf:
        for j in unrecovered_regs:
            recovered[j] = best_states_bf[j]
            print(f"  R{j+1}: {recovered[j]} (0b{recovered[j]:010b})")

print(f"\n*** All recovered initial states: ***")
for j in range(n):
    if j in recovered:
        print(f"  R{j+1}: {recovered[j]:4d} (0b{recovered[j]:010b})")
    else:
        print(f"  R{j+1}: UNKNOWN")

# ============================================================
# Step 11. Verification
# ============================================================
# Generate gamma from the recovered initial states using the full
# combining function f, and compare with the input gamma bit by bit.

print(f"\n{'='*60}")
print("Step 11. Verification")
```

```python
print(f"{'='*60}")

if len(recovered) < n:
    print("Cannot verify: not all registers recovered.")
else:
    # Try both offset conventions for verification
    for off_name, off in [("offset=0", 0), ("offset=1", 1)]:
        reg_states = [recovered[j] for j in range(n)]
        states_v = list(reg_states)
        mask_m = (1 << m) - 1

        generated = []
        for t_idx in range(N + 2):
            # Evaluate f on current outputs (LSB of each register)
            f_input = 0
            for j in range(n):
                if states_v[j] & 1:
                    f_input |= (1 << j)
            generated.append(f_tt[f_input])

            # Clock all registers
            for j in range(n):
                fb = bin(states_v[j] & tap_masks[j]).count('1') & 1
                states_v[j] = ((states_v[j] >> 1) | (fb << (m - 1))) & mask_m

        # Compare with gamma (using offset)
        match_count = sum(1 for i in range(N) if gamma[i] == generated[i + off])
        pct = float(100 * match_count / N)
        print(f"  {off_name}: {match_count}/{N} ({pct:.2f}%)")

        if match_count == N:
            print(f"  *** VERIFICATION PASSED ({off_name})! ***")
            break

# ============================================================
# Summary
# ============================================================
print(f"\n{'='*60}")
print("Summary")
print(f"{'='*60}")
print(f"Combining function: f = {f_poly}")
print(f"deg(f) = {f_poly.degree()}, wt(f) = {wt_f}/{2**n}")
print(f"cor(f) = {cor_f}")
print(f"Number of affine analogs: {len(analogs)}")
print(f"Gamma length: {N}")
print(f"delta = {delta}")
print(f"LFSR length: m = {m}")
print(f"Number of LFSRs: n = {n}")
print(f"Precomputation time: {t_precomp:.2f} s")
print(f"\nRecovered initial states:")
```

```
for j in range(n):
    if j in recovered:
        print(f"  R{j+1}: {recovered[j]:4d} (0b{recovered[j]:010b})")
```

Список використаних джерел

[1] Thomas Siegenthaler. «Decrypting a Class of Stream Ciphers Using Ciphertext Only». In: *IEEE Transactions on Computers* C-34.1 (1985), pp. 81–85.

[2] Claude Carlet. *Boolean Functions for Cryptography and Coding Theory*. Cambridge: Cambridge University Press, 2021.